Adaptive KV Materialization for Multi-GPU vLLM Serving

Prototype Draft

Abstract

Large language model serving systems increasingly rely on reusable KV and prefix state, but realizing that state is not free. This draft studies a single policy problem: when reusable state exists for a newly arrived request, should the system fully reuse it, partially materialize it, or recompute the prefix from scratch? We structure the repository around an experimental study first and a future vLLM plugin second.

1 Problem

Existing serving work shows that reusable state can reduce prompt processing and improve latency, but reuse realization is not free. When state lives remotely, needs transfer, or requires nontrivial stitching, aggressive reuse can increase time to first token rather than reduce it. The practical question is therefore not whether reuse exists, but how much reusable state should be materialized at request arrival.

2 Hypothesis

The best materialization action depends on reuse volume, transfer cost, queue pressure, and request latency sensitivity.

3 Policy Surface

This repository studies a single-hook decision problem at request arrival. Given reusable state, the controller chooses one of three actions:

- `full_reuse`: materialize the full reusable prefix,
- `partial_reuse`: materialize only a prefix segment and recompute the rest,
- `recompute`: ignore reusable state and run the prompt path from scratch.

This keeps the study orthogonal to placement-policy work and orthogonal to eviction-policy work.

4 Offline Study Snapshot

The current repository ships a workload-driven offline study harness. On three representative cases derived from the shared `llm-serving-workloads` repository, the heuristic policy achieves mean TTFT 8.411ms and p95 TTFT 22.161ms. The oracle action selector reaches mean TTFT 8.384ms and p95 TTFT 22.161ms, which gives a useful upper bound on the remaining policy headroom while keeping the action space fixed to full reuse, partial reuse, and recompute.

Table 1: Offline study snapshot on workload-driven traces derived from llm-serving-workloads.

Policy	Mean TTFT	P95 TTFT	Mean recompute tok	Mean transfer KiB
always recompute	12.847	24.286	620.76	0.0
always full reuse	8.384	22.161	0	19864.333
threshold partial	11.371	22.161	384.344	7565.333
heuristic	8.411	22.161	1.083	19829.667
oracle ttft	8.384	22.161	0	19864.333

5 Plan

We compare full reuse, partial reuse, and recompute across workload families with strong state locality. The current repository step is workload-carrier evaluation through the shared `llm-serving-workloads` repository, with oracle and cost-misestimation sensitivity baselines added around the main heuristic. The next step is to wire the same decision interface into a vLLM plugin hook and validate online TTFT behavior.

6 Initial Contributions

1. A standalone repository that packages the policy, experiment harness, and paper artifacts together.
2. A first offline study pipeline that emits paper-consumable summaries.
3. A narrow plugin target centered on request-arrival materialization instead of placement or eviction.

References